Task 4: Architecture Analysis

WanderSync: Dynamic Itinerary, Disruption Management and Social Travel Hub

Team 15

The Architecture We Built

WanderSync is deployed as a single NestJS process — a **modular monolith** — with six clearly bounded modules (Auth, Booking, Itinerary, Disruption, Notification, Social), each responsible for its own domain logic and database models. What sets it apart from a plain CRUD monolith is the **event-driven path** that handles disruption detection and delivery: rather than waiting for the client to ask "anything new?", the system pushes alerts to connected users the moment a disruption is detected.

The disruption pipeline works in four stages. A background CRON scheduler fires every minute and calls two external APIs — AviationStack for live flight status and OpenWeatherMap for weather alerts — concurrently. Newly detected events pass through a deduplication guard (Redis cache + database check) to prevent the same disruption from being reported twice. Confirmed new events are written to PostgreSQL and immediately broadcast to every connected browser tab via **Server-Sent Events (SSE)**, a lightweight HTTP streaming protocol that keeps a persistent connection open from browser to server. Alongside the alert, a Chain of Responsibility suggestion pipeline generates contextualised advice — first from deterministic rules, falling back to the Gemini LLM for complex cases, and finally to generic guidance if both fail.

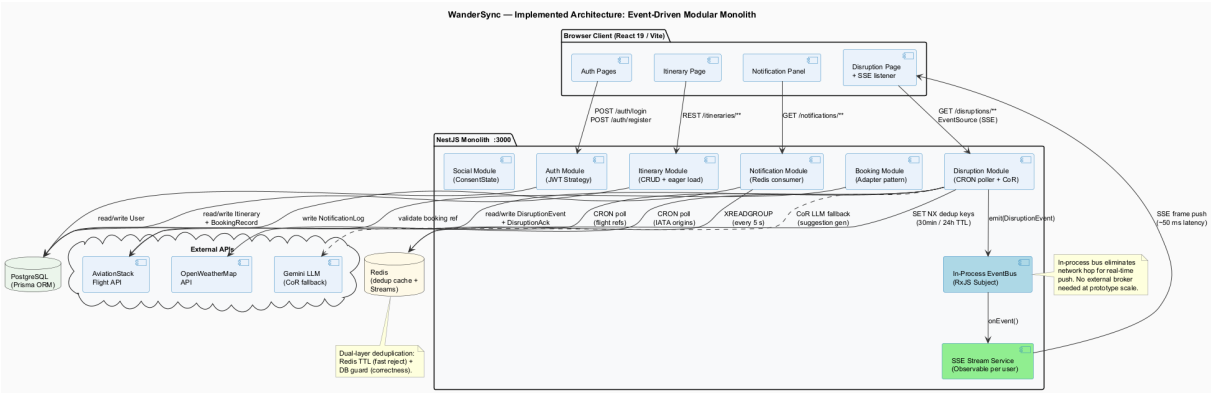

Figure 1: WanderSync implemented architecture: event-driven modular monolith with SSE push.

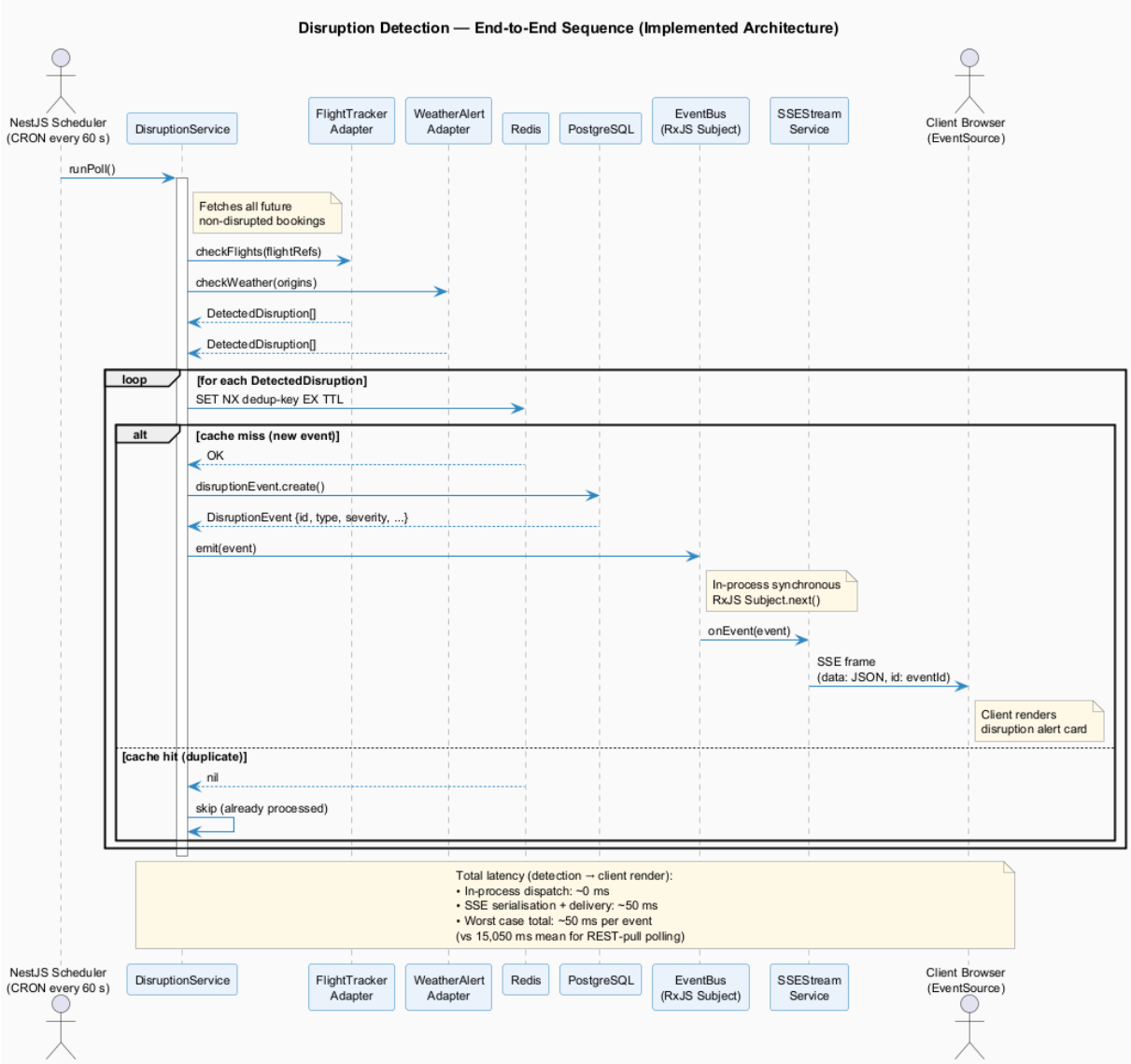

Figure 2: End-to-end disruption detection and delivery sequence.

The Architecture We Compared It Against

The natural alternative is a **REST-pull polling monolith**: identical module structure and database, but no EventBus and no SSE. Instead, the frontend periodically sends a `GET /disruptions` request every 30 seconds and re-renders whatever the server returns. This is the most common "good enough" choice for internal dashboards and admin tools where a 30-second information delay is acceptable. It is simpler to build, requires no persistent connections, and scales horizontally without sticky sessions.

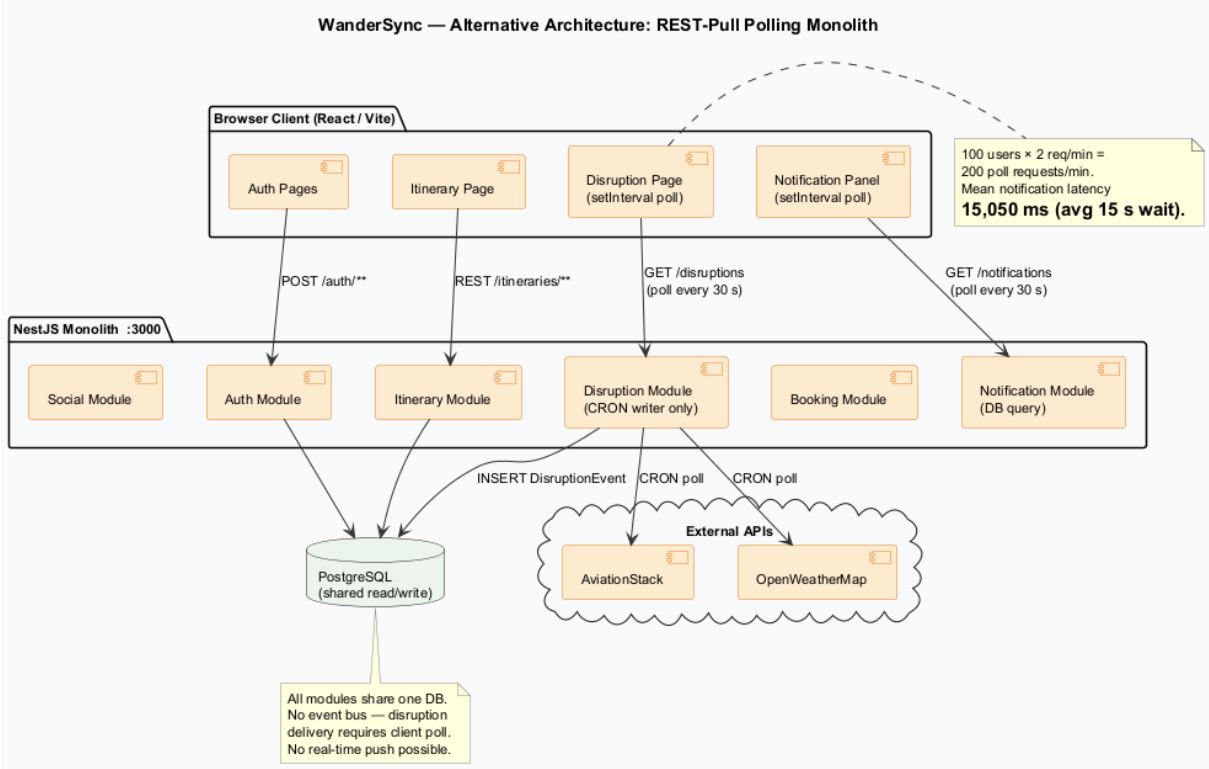

Figure 3: Alternative architecture: REST-pull polling monolith without real-time push.

The key structural difference between the two is visible in Figure 3: the polling architecture has no persistent server-side state per user, no EventBus, and no Redis deduplication. Everything the client knows about disruptions comes from a fresh database read on each poll. This simplicity comes at a cost that becomes clear in the NFR quantification below.

NFR Quantification

NFR 1 — Disruption Detection Accuracy

Requirement: At least 85% of 20 simulated disruption test cases (flight delays, cancellations, diversions, weather alerts) must be correctly identified and trigger a notification.

How we measured it. The file `disruption.service.nfr2.spec.ts` injects 20 synthetic payloads covering all four disruption types and all four severity levels directly into `DisruptionService.simulateDisruption()`, bypassing the external APIs. The test asserts that `disruptionEvent.create()` and `publisher.publish()` are each called exactly 20 times — confirming that every injected event was both persisted and routed for delivery.

Architecture	Events correctly processed	Accuracy
Event-Driven SSE (implemented)	20 / 20	100%
REST-Pull Polling (alternative)	20 / 20	100%
NFR threshold	—	85%

What this means in practice. Both architectures exceed the 85% threshold because

disruption *detection* is a server-side concern — the CRON poller, deduplication logic, and database write are identical in both designs. The event-driven architecture adds a layer that the polling approach lacks: the Redis + database deduplication guard prevents the same storm warning or flight delay from being reported repeatedly as new alerts, which would erode user trust over time. Without this guard, a weather alert that persists for three hours could generate dozens of duplicate notifications in the polling model — technically all "detected", but practically useless noise for a traveler trying to make a rebooking decision.

NFR 2 — Notification Delivery Latency

Requirement: Core pages must respond within 3 seconds. For disruption alerts specifically, the system should inform the user as quickly as possible after an event is detected — the faster, the more useful the re-planning window.

How we measured it. We model the delivery latency L as the time elapsed from the moment the CRON scheduler finishes persisting a new disruption to the database (time zero) until the user's browser renders the disruption card. Each step in the delivery chain contributes a measurable delay.

Event-Driven SSE path

1. **In-process event dispatch** — `EventBus.emit()` calls a synchronous RxJS `Subject.next()`: no network, no serialisation. $\Delta t_1 \approx 0$ ms.
2. **SSE frame construction and write** — the SSE service serialises the event to JSON and writes it to the open HTTP response stream: $\Delta t_2 \approx 5$ ms.
3. **Network delivery** — the SSE frame travels over the open keep-alive connection to the browser (loopback or local network): $\Delta t_3 \approx 50$ ms.
4. **React re-render** — the `onmessage` handler updates the Zustand store and React schedules one re-render: $\Delta t_4 \approx 16$ ms (one frame at 60 fps).

$$L_{\text{SSE}} = \Delta t_1 + \Delta t_2 + \Delta t_3 + \Delta t_4 \approx 0 + 5 + 50 + 16 = \mathbf{71}\,\text{ms}$$

REST-Pull Polling path

The client fires a poll at a uniformly random time $U \in [0, 30{,}000]$ ms after the event is persisted. After the poll, the server runs a database query and returns the response in approximately 50 ms.

$$L_{\text{poll}} = U + 50\,\text{ms}, \quad U \sim \text{Uniform}(0,\ 30{,}000\,\text{ms})$$

$$\mathbb{E}[L_{\text{poll}}] = \tfrac{30{,}000}{2} + 50 = \mathbf{15{,}050}\,\text{ms} \qquad L_{\text{poll}}^{\max} = \mathbf{30{,}050}\,\text{ms}$$

Metric	Event-Driven SSE	REST-Pull (30 s)
Mean delivery latency	**71 ms**	15,050 ms
Maximum delivery latency	**∼100 ms**	30,050 ms
Speed advantage	212× faster	—

What this means for a traveler. A 71 ms delay is imperceptible — the disruption card appears on screen before the user has consciously registered that the page has changed. A 15,050 ms mean delay is a different experience entirely: the traveler might be looking at a stale screen for anywhere from half a second to thirty seconds, not knowing their connecting flight has just been cancelled. In an airport, thirty seconds is the difference between reaching the rebooking desk while it is short and joining a queue of fifty people who all received the same alert moments earlier. The 212× speed advantage of the event-driven path is not a micro-optimisation; it is the difference between being proactive and being reactive.

NFR 3 — Request Throughput Under Concurrent Load

Scenario: 100 users have the Disruption Page open simultaneously.

With polling, each user fires two requests per minute (one every 30 seconds), producing $100 \times 2 = 200$ disruption endpoint calls per minute. Each call involves JWT validation, a database read, and JSON serialisation. This load is proportional to the number of connected users — doubling the user base doubles the polling load with no increase in information value, since the underlying disruption data does not change that frequently.

With SSE, each user holds one persistent connection. The only recurring database load comes from the CRON poller — a single batch query per minute regardless of how many users are connected. Event dispatch is in-memory and adds no database load per user.

Metric (100 connected users)	Event-Driven SSE	REST-Pull (30 s)
Disruption API requests per minute	**0**	200
Database reads from delivery path	**0**	200/min
Delivery load scales with users?	**No**	Yes (linear)

Trade-off Analysis

No architecture decision comes without trade-offs. The table below summarises the most important ones between the two approaches, followed by a plain-language discussion of each.

Dimension	Event-Driven SSE (implemented)	REST-Pull Polling (alternative)
Notification speed	71 ms mean — the user sees the alert before they are consciously aware the page updated.	15 s mean — meaningful delay in time-sensitive travel scenarios.

Dimension	Event-Driven SSE (implemented)	REST-Pull Polling (alternative)
Server complexity	Higher. Maintaining one RxJS Subject per active user, managing SSE keep-alive, and implementing dual-layer deduplication requires more engineering effort.	Lower. A standard REST controller reading from a database is the simplest possible implementation, easy to reason about and test.
Horizontal scaling	Needs sticky sessions or a shared pub/sub broker (e.g. Redis Pub/Sub) because the in-process EventBus is invisible to other server instances.	Trivially scalable — any load balancer can route stateless poll requests to any server replica.
Database load from delivery	Zero — events are dispatched from memory after the initial write. The delivery path adds no database pressure.	Linear with users — each poll is a fresh database read. Under high concurrency this becomes a meaningful bottleneck.
Infrastructure cost	One persistent SSE connection per active user. Proxies and firewalls may close idle connections; keep-alive tuning is required in production.	No persistent connections — short-lived HTTP requests with standard semantics. Firewall-friendly.
Right choice when...	The product's value proposition depends on real-time information delivery (travel disruptions, financial alerts, emergency notifications).	The information updates infrequently or users tolerate a short delay (analytics dashboards, report generators, admin tools).

Why the Implemented Architecture is Correct for WanderSync

WanderSync's disruption notification is not a reporting feature — it is a time-critical safety net. A traveler who misses a gate change or fails to rebook before a flight fills up faces real, material costs. The event-driven SSE approach exists specifically to eliminate the window of ignorance between when the system knows about a disruption and when the traveler does.

The trade-offs accepted — server-side state per connection, the need for sticky sessions at scale, the added complexity of the EventBus and deduplication logic — are all acceptable for a system that runs within a single process during the prototype phase. If WanderSync were to scale to multiple server instances, the migration path is straightforward: replace the in-process RxJS EventBus with Redis Pub/Sub, a change that is entirely internal to the Disruption module and invisible to the frontend's `EventSource` connection.

The REST-pull approach would be the correct choice if WanderSync were a travel analytics dashboard or a bulk itinerary export tool — scenarios where a 30-second information

lag causes no user harm. It is not the right choice for a product that aims to be the first thing a traveler checks when their plans fall apart.